Quo Vadis, Code Review? Exploring the Future of Code Review

Michael Dorner
Technische Hochschule Nürnberg Georg Simon Ohm
Nürnberg, Germany
michael.dorner@th-nuernberg.de

Andreas Bauer
Technische Hochschule Nürnberg Georg Simon Ohm
Nuernberg, Germany
andreas.bauer@th-nuernberg.de

Darja Šmite
Blekinge Institute of Technology
Karlskrona, Sweden
darja.smite@bth.se

Lukas Thode
Blekinge Institute of Technology
Karlskrona, Sweden
lukas.thode@bth.se

Daniel Mendez
Blekinge Institute of Technology
Karlskrona, Sweden
fortiss
München, Germany
daniel.mendez@bth.se

Ricardo Britto
Blekinge Institute of Technology
Karlskrona, Sweden
Ericsson
Karlskrona, Sweden
ricardo.britto@bth.se

Stephan Lukasczyk
JetBrains Research
Munich, Germany
stephan.lukasczyk@jetbrains.com

Ehsan Zabardast
Blekinge Institute of Technology
Karlskrona, Sweden
ehsan.zabardast@bth.se

Michael Kormann
SAP
Walldorf, Germany
michael.kormann@sap.com

Abstract

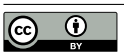
Context: Code review has long been a core practice in collaborative software engineering. As automation becomes increasingly embedded in development workflows, the role and functioning of code review are subject to change.

Objective: This study explores how professional developers anticipate the evolution of code review and identifies emerging tensions reflected in these expectations.

Method: We conducted a cross-sectional survey with 100 developers across five software-driven companies. The survey captured estimates of current review time and reviewed artifacts, as well as anticipated changes over a five-year horizon. Open-ended questions invited reflections on the future of code review. Quantitative responses were analyzed descriptively, and open-ended responses were independently coded by multiple researchers using thematic analysis to identify recurring patterns in participant responses.

Results: Practitioners expect code review to remain essential, anticipating stable or increased time investment and a broader range of reviewed artifacts over the next five years. In open-ended responses, many participants explicitly referenced AI and large language models (LLMs), describing increasing automation in both code authoring and reviewing, including scenarios in which automated systems operate in both roles.

Conclusion: Our analysis suggests emerging tensions concerning understanding, accountability, and trust in automation-mediated code review. These tensions provide early empirical signals of sociotechnical challenges and position code review as a concrete setting for examining the implications of LLM integration in collaborative software engineering.



EASE '26, Glasgow, United Kingdom

ACM ISBN 979-8-4007-2348-3/26/06
https://doi.org/10.1145/3816483.3816556

CCS Concepts

• **Software and its engineering** → **Collaboration in software development**; *Software verification and validation*; • **Computing methodologies** → Artificial intelligence.

Keywords

code review, large language models, generative AI, human–AI collaboration, collaborative software engineering

ACM Reference Format:
Michael Dorner, Andreas Bauer, Darja Šmite, Lukas Thode, Daniel Mendez, Ricardo Britto, Stephan Lukasczyk, Ehsan Zabardast, and Michael Kormann. 2026. Quo Vadis, Code Review? Exploring the Future of Code Review. In *International Conference on Evaluation and Assessment in Software Engineering (EASE '26), June 09–12, 2026, Glasgow, United Kingdom.* ACM, New York, NY, USA, 6 pages. https://doi.org/10.1145/3816483.3816556

1 Introduction

During the early days of software engineering, code was often designed and implemented by individual developers. As software systems became more complex, software engineering evolved into a collaborative effort involving multiple developers with diverse skills, organized in teams. To maintain a shared understanding of the codebase and its changes, developers discuss code changes before they are integrated.

As software engineering scaled from individual to collaborative and globally distributed work, these discussions evolved accordingly. In the 1970s, they were structured, in-person, and formal, and known as *code inspections* [12]. In the 1990s and 2000s, *pair programming* integrated a synchronous form of those discussions between two developers directly into development workflows [17]. As development became increasingly distributed, discussions shifted toward asynchronous, tool-supported formats. Today, they are commonly referred to as *code reviews*, a widely adopted practice in collaborative software engineering [6, 11].

While prior research has predominantly framed the practice as human-centered and collaborative [3, 6, 11], automation in software engineering has expanded steadily. Recent advances in large language models (LLMs) have accelerated this trajectory. Empirical studies have begun evaluating the quality and usefulness of LLM-generated review comments [1], and practitioner-oriented reports document active exploration of generative AI tools in code review contexts [9]. At the same time, LLMs are increasingly used for code generation [5], raising the possibility that automation may extend into both authoring and reviewing. This shift raises new questions about how the role and configuration of code review may evolve.

The objective of this study is to explore how professional developers anticipate the evolution of code review and identify emerging tensions reflected in these expectations.

To address this objective, we conducted a cross-sectional survey with 100 across five companies. As changes in code review cannot be directly observed within such a design, we operationalized them through participants' reported current practices and anticipated changes in time spent on code review, reviewed artifacts, and expected changes to code review and their implications over the next five years.

This study contributes (1) empirical evidence on anticipated evolution of code review, (2) a conceptual model of human–AI role configurations, and (3) an analysis of emerging tensions in understanding, accountability, and trust.

The remainder of this paper presents the survey design and analysis approach (Section 2), reports empirical findings on anticipated evolution of code review (Section 3), analyzes emerging tensions arising from these findings (Section 4), and concludes with implications for research and practice (Section 6).

2 Method

We conducted an exploratory cross-sectional survey to examine how professional software developers perceive code review today and how they anticipate its evolution over the next five years.

2.1 Sampling Strategy

Because no comprehensive sampling frame of professional software developers exists [4], we employed *quota sampling*, a non-random two-stage sampling strategy. First, we purposively selected five companies representing diverse industry domains, primary software system types, and organizational scales. Second, we aimed to recruit approximately 25 developers per company. Recruitment concluded after eight weeks or once the target quota was reached.

2.2 Sample

The final sample comprises 100 professional software developers from five companies, including SAP, Ericsson, JetBrains, Gradle, and one large European bank that requested anonymity (see Table 1). All respondents reported involvement in code review within their organizations. Participation varied by company due to voluntary recruitment. Responses are analyzed at the aggregate level, where companies serve as contextual background rather than analytical units.

2.3 Survey Instrument

The survey captured both perceptions of current code review and anticipated changes over a five-year horizon. The five-year timeframe was selected to balance near-term plausibility with sufficient scope for structural change.

To capture participants' perceptions of current code review practices, we collected self-reported data on:

- Average weekly time spent on code review (numerical response)
- Types of software artifacts currently reviewed (multiple selection)

To capture anticipated evolution, participants indicated:

- Whether they expect to spend more, less, or the same amount of time on code review in five years (closed-end)
- Which artifacts they anticipate reviewing in five years (multiple selection).
- What major changes they anticipate in code review (open-ended)
- What implications they foresee based on those changes (open-ended)

Artifact categories included production code, test code, configuration files, documentation, and GUI-based test code. Participants could add additional artifact types.

2.4 Data Collection

The survey was conducted between December 2024 and November 2025. For each company, we set up an independent instance of the online questionnaire and distributed it through the company's internal communication channels. Recruitment was conducted sequentially across organizations, with each company-specific survey window remaining open for up to eight weeks or until the targeted quota was reached. Participation was voluntary and anonymous. To preserve confidentiality, we did not collect demographic information such as age, gender, or role seniority. Participation counts per company are reported without attributing responses to individuals or specific organizations.

2.5 Data Analysis

Quantitative responses were analyzed descriptively to summarize current time investment and reviewed artifacts, as well as anticipated changes over the five-year horizon.

Open-ended responses were analyzed using a reflexive form of thematic analysis, following the approach outlined by Clarke and Braun [8]. The first author conducted inductive descriptive coding. Initially, all responses were reviewed to ensure familiarity with the data. Preliminary codes were then assigned to participants' descriptions of anticipated changes and perceived implications. Codes were iteratively refined as additional responses were analyzed.

Subsequently, codes were examined and grouped into broader themes based on conceptual or semantic similarities. A theme captures "something important about the data in relation to the research question, and represents some level of patterned response or meaning within the data set" [7].

The second author reviewed the codes to develop broader themes. Consistent with reflexive thematic analysis, we moved away from

Table 1: Companies of the participating developers and sample sizes per company

Attribute	SAP	Ericsson	*A Bank*	JetBrains	Gradle
Domain	Enterprise software	Telecommunications	Banking and finance	Developer Tools	Developer Tools
Software System Type	Customer-facing enterprise platforms	Embedded and network control software	Internal financial transaction and risk management systems	Developer productivity and language tooling	Build automation and developer productivity tooling
# of employees	>100,000	>100,000	>25,000	>2,200	>150
# of participants	31	24	25	11	9

independent parallel coding or inter-rater reliability in favor of collaborative discussion. Throughout this process, we drew on our professional backgrounds in empirical software engineering to reflexively interpret the data through a socio-technical lens. Rather than reporting the numerical prevalence of themes, we concentrated on the conceptual importance across all responses to ensure that findings like accountability and trust were grounded in both participant reflections and professional context.

3 Results

This section presents descriptive findings on (1) current weekly time spent on code review and expected changes over a five-year horizon, (2) reviewed artifacts today and anticipated future coverage, and (3) qualitative patterns emerging from open-ended responses regarding anticipated changes and their implications. All findings reflect self-reported perceptions and expectations rather than observed longitudinal developments.

3.1 Effort and Scope of Code Review

Across 100 respondents, the median self-reported time spent on code review is approximately three hours per week (IQR: 2–6). Most respondents report spending fewer than five hours per week, while a smaller subset reports substantially higher time investment of up to 16 hours.

When asked how their review time may change over the next five years, 47 % respondents expect to spend *more time*, 30 % expect to spend *about the same time*, and 23 % expect to spend *less time* on code review.

Respondents also anticipate a broader scope of reviewed artifacts. Figure 1 shows paired percentages for artifacts reviewed *today* versus those expected *in five years*. Production code remains dominant (86% today; 89% expected), while increases are anticipated for test code, configuration files, documentation, and GUI-based test code. The share reporting that they review no artifacts decreases slightly.

Taken together, responses indicating stable or increasing review effort substantially outnumber those anticipating a decrease. Similarly, expectations of an expansion in reviewed artifacts exceed those anticipating contraction.

3.2 AI as a Participant in Code Review

In open-ended responses on the anticipated changes, many participants explicitly referenced AI and LLMs, even though the survey did not explicitly focus on AI. Rather than replacing human reviewers entirely, participants describe configurations in which AI performs early filtering or routine assistance before human inspection.

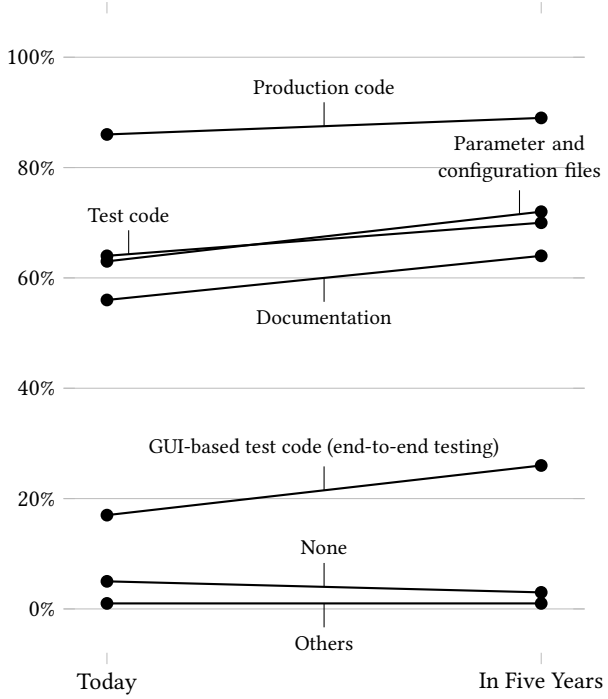


Figure 1: Expected change in reviewed artifacts: respondents expect broader review coverage, with the largest increase for GUI-based test code. Percentages reflect multiple selection.

For example, one respondent noted that *"AI automated review will become standard to identify potential pitfalls automatically – probably before the human review happens."* Another suggested *"Initial code review to be done by some form of an AI that can pick some basic stuff that are often missed by users."* Similarly, participants described minor issues being *"sorted out with help of AI tools well before code review"* and AI performing a *"pre-review, e.g., highlighting the important points or adding explanations."*

In these accounts, AI is expected to handle syntactic checks, common defects, and routine issues, while human reviewers focus on

architectural decisions, domain logic, and solution-level concerns. Several respondents explicitly anticipated a shift away from style-level discussions toward higher-level reasoning, with one stating that review would focus *"more on architectural aspects instead of code style aspects."*

These responses point to a shift in the distribution of responsibilities within code review, where automation supports routine checks and human reviewers remain central to higher-level evaluation.

3.3 More Code to Review

Beyond the integration of AI into code review, many participants anticipate a substantial increase in the volume of generated code and pull requests due to AI-assisted coding. This increase is expected to raise the overall demand for code review.

A respondent observed that *"AI will produce a tons of code with various quality. Reading that code will be more important than actually writing it."* Another anticipated *"an overwhelming amount of reading that we'll have to do (less writing and tons more of reading)."* Others explicitly connected AI-assisted coding with increased review workload, noting that *"more code to review because of AI assisted coding"* and *"spending more time on code review rather than coding."*

These responses suggest that AI-assisted coding may not reduce code review effort, but instead increase the volume of code requiring reviews. Therefore, developers anticipate spending more time assessing and validating generated code. This expectation aligns with the quantitative finding that stable or increasing review time substantially outweighs expectations of decline.

3.4 Fully Automated Configurations

While many responses describe hybrid configurations, some participants explicitly reflected on the possibility of simultaneous AI authoring and reviewing. These reflections articulate a boundary scenario in which both code generation and code review are delegated to automated systems.

One participant cautioned: *"We will have to decide if we want to write code with AI or review code with AI [...] I don't think both at the same time will work [...] the AI doesn't catch the mistake because it generated it."* Another described the prospect of *"AI will be used to review code, and then we will have to review code and also the AI review of the code."*

Such responses reflect concerns about potential feedback loops and error propagation when automated systems operate in both authoring and reviewing roles. We conceptualize these role combinations along two intersecting continua: code author (human–LLM) and code reviewer (human–LLM). As illustrated in Figure 2, this yields four possible role combinations, including a fully automated boundary case. We present this boundary case as a conceptual extreme derived from participant reflections rather than as a prediction of near-term practice.

3.5 Human Oversight and Responsibility

Despite expectations of increasing automation, multiple participants emphasized that human review will remain essential. Respondents did not describe a future without code review, but one in which AI participation coexists with continued human responsibility.

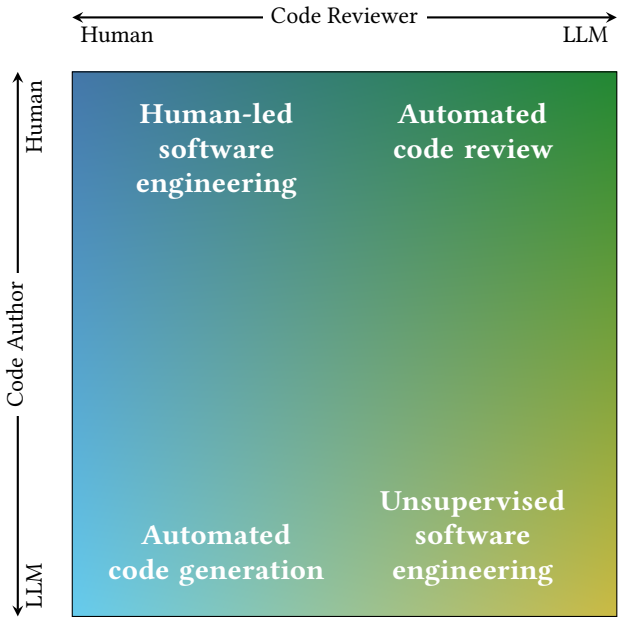


Figure 2: Code review conceptualized along two continuous dimensions: author and reviewer roles ranging from fully human-led to fully LLM-led. The resulting quadrants represent alternative configurations of human–AI collaboration suggested by participant responses.

For example, one participant stated, *"I do not think that code review will disappear completely due to agents or LLMs."* Another argued that *"human involvement will be much more important,"* particularly when AI-generated code is involved. Several responses also referenced legal and organizational accountability, suggesting that AI-generated code would still require human review to ensure responsibility, including to *"avoid any [lawsuits] on the code development organization."*

These responses indicate that, even in settings with substantial automation, practitioners expect human oversight to remain central to code review. While AI may assist with routine checks or preliminary analysis, responsibility for evaluating correctness, domain relevance, and organizational risk is expected to remain with human reviewers.

4 Discussion

Our findings indicate that practitioners do not expect code review to decline. Instead, they anticipate stable or increasing time investment, broader artifact coverage, and a growing volume of AI-generated code requiring review. At the same time, participants expect increasing integration of AI, typically in supportive roles rather than as full replacements for human reviewers. We interpret those insights as early empirical signals of change in code review and discuss their implications along three dimensions of code review: understanding, accountability, and trust.

4.1 Understanding

Several respondents expressed concern that increasing reliance on AI-generated code may alter how developers understand the systems they maintain. Expectations of reviewing larger volumes of generated artifacts and spending more time reading than writing suggest a shift in how developers engage with code.

Code review has traditionally served not only as a quality assurance mechanism, but also as a communication network and collaborative practice [11, 13, 14]. In layered configurations where AI performs preliminary checks and developers focus on higher-level concerns, the nature of understanding may shift from line-by-line scrutiny toward validation of generated solutions. This shift does not necessarily imply a decline in comprehension. However, it may change how, where, and potentially even whether the same depth of understanding is developed within human teams.

Concerns about diminishing experiential learning were also visible in responses suggesting that AI-assisted generation may reduce opportunities for developers to engage deeply with implementation details. Such expectations resonate with recent discussions of AI-generated code as a form of generative reuse [16], in which developers increasingly integrate artifacts they did not design. If the anticipated scale increase materializes, maintaining long-term human comprehension may require deliberate practices to keep pace with the growing volume of generated artifacts.

4.2 Accountability

Despite expectations of increasing automation, respondents repeatedly emphasized the continued necessity of human oversight. Some explicitly referred to legal exposure and organizational responsibility, suggesting that AI-generated code would remain subject to human review to ensure accountability.

Code review has long functioned as a mechanism for distributing and reinforcing responsibility within teams [3, 18]. In layered human–AI configurations, however, evaluative work may be partially delegated to tools that cannot themselves bear responsibility. While human developers remain formally accountable, the evaluative authority may become less clearly defined when AI generates code, highlights defects, or proposes fixes.

Rather than eliminating accountability, automation may redistribute it. Human reviewers may increasingly evaluate not only code changes, but also AI-generated suggestions and analyses. This dual evaluation structure, reflected in responses describing the need to *"review the AI review"*, suggests a potential expansion of oversight responsibilities. As regulatory frameworks increasingly emphasize traceability and governance in software engineering [10, 15], ensuring transparent lines of human responsibility in AI-augmented code review may become increasingly critical.

4.3 Trust

Trust emerged implicitly in several responses reflecting skepticism toward fully automated configurations. Participants questioned the simultaneous delegation of authoring and reviewing to AI, highlighting concerns about feedback loops and error propagation. The scenario in which AI-generated code is subsequently reviewed by another AI was described as potentially problematic, particularly if errors introduced during generation remain undetected.

Trust in code review traditionally rests on interpersonal knowledge, shared context, and identifiable contributors. In AI-augmented settings, distinguishing between human judgment and automated suggestion may become less straightforward. As one participant noted, the involvement of AI may require additional scrutiny rather than less, potentially making review more complex rather than simpler.

Empirical studies have already shown that developers assess AI-generated review comments differently from human feedback [2]. Our findings suggest that such distinctions may become increasingly relevant as AI participation expands. Maintaining calibrated trust in hybrid human–AI review processes may therefore require clearer attribution of contributions, explicit norms governing AI use, and transparency about tool involvement.

5 Threats to Validity

As an exploratory mixed-methods study, our research design involves inherent trade-offs. A primary limitation is our reliance on self-reported perceptions of the future. To mitigate the highly speculative nature of this data, we operationalized anticipated evolution through concrete, comparative assessments, such as current versus anticipated time spent and artifacts reviewed, rather than asking for open-ended predictions of the distant future.

Consequently, a key limitation is that our findings reflect practitioners' expectations rather than observed changes. While such perceptions offer early signals, they are inherently speculative and may be influenced by the recent prominence of generative AI. Therefore, results should be interpreted as indicative of perceived trajectories, not validated developments.

Another notable threat stems from our sequential data collection across the five participating companies from December 2024 to November 2025. While this extended period was a logistical necessity to accommodate organizational communication policies, it coincided with rapid advancements in generative AI tools. However, our analysis revealed consistent thematic patterns regarding core tensions (understanding, accountability, and trust) across early and late respondents. This suggests that practitioners' long-term expectations are anchored in fundamental collaborative practices rather than short-term fluctuations in tool capabilities.

Furthermore, because no comprehensive sampling frame of software developers exists, we employed non-random quota sampling. While our sample of 100 developers is not statistically representative of the global software engineering population, our purposive selection of companies representing distinct industry domains and varying scales ensures a diverse baseline of practitioner perspectives, supporting the transferability of our findings to similar environments.

Finally, our qualitative findings rely on the thematic analysis of open-ended responses and are inherently subject to researcher subjectivity. Consistent with reflexive thematic analysis, we did not attempt to eliminate this subjectivity via independent parallel coding. Instead, we leveraged our backgrounds in empirical software engineering to collaboratively and reflexively interpret the data, ensuring themes were grounded directly in participant accounts rather than preconceived notions about automation.

6 Conclusion

Code review has long been a cornerstone of collaborative software engineering, where experience, reasoning, and feedback shape the evolution of software systems. Our results indicate that it is unlikely to disappear in the foreseeable future. On the contrary, participants expect stable or increasing review effort and an expansion in the range of reviewed artifacts.

While code review will remain essential, it is expected to evolve substantially. In particular, participants anticipate that LLMs will become ubiquitous in development workflows and increasingly take on tasks in both code authoring and review. This raises the question to what extent evaluative responsibilities should be delegated to AI or remain with human developers. Rather than assuming purely beneficial effects, our findings point to emerging concerns regarding over-reliance on LLMs, including potential impacts on understanding, accountability, and trust.

As software engineering continues to evolve toward greater automation, we therefore advocate for a nuanced perspective on code review. While automation may improve efficiency, maintaining human understanding, accountability, and calibrated trust remains critical for sustaining control over the long-term evolution of software systems. These qualities, although less easily measurable or optimizable, underpin the resilience of software engineering in the face of ongoing technological change.

We also encourage further research into the implications of integrating LLMs and other generative AI techniques into code review, as this practice may serve as an early indicator of how AI reshapes collaborative software engineering more broadly. An over-reliance on AI may risk shifting the locus of understanding from humans to the LLM—an entity that, despite its capabilities, cannot be held accountable, reason contextually, or justify its decisions beyond probabilistic associations. To preserve the integrity of software engineering as a collaborative, transparent, trustworthy, and responsible engineering practice, it remains essential that code review remains grounded in human understanding and oversight.

Data and Code Availability

The survey instrument, all anonymized survey responses and analysis scripts, as well as coding materials, are publicly available at: https://github.com/michaeldorner/quo-vadis-code-review

Acknowledgments

We thank all participants of the survey and the anonymous reviewers for their insightful and constructive feedback. This work was supported by the KKS Foundation through the SERT Project (Research Profile Grant 2018/010) at Blekinge Institute of Technology.

References

[1] Adam Alami, Nathan Cassee, Thiago Rocha Silva, Elda Paja, and Neil A. Ernst. 2025. Engagement in Code Review: Emotional, Behavioral, and Cognitive Dimensions in Peer vs. LLM Interactions. *arXiv preprint arXiv:2512.05309* (12 2025).

[2] Adam Alami and Neil Ernst. 2025. Human and Machine: How Software Engineers Perceive and Engage with AI-Assisted Code Reviews Compared to Their Peers. In *2025 IEEE/ACM 18th International Conference on Cooperative and Human Aspects of Software Engineering (CHASE)*. IEEE, 63–74.

[3] Alberto Bacchelli and Christian Bird. 2013. Expectations, outcomes, and challenges of modern code review. In *2013 35th International Conference on Software Engineering (ICSE)*. 712–721. doi:10.1109/ICSE.2013.6606617

[4] Sebastian Baltes and Paul Ralph. 2022. Sampling in software engineering research: a critical review and guidelines. *Empirical Software Engineering* 27 (7 2022), 94. Issue 4. doi:10.1007/s10664-021-10072-8

[5] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7 (4 2023), 85–111. Issue OOPSLA1. doi:10.1145/3586030

[6] Andreas Bauer, Riccardo Coppola, Emil Alégroth, and Tony Gorschek. 2023. Code review guidelines for GUI-based testing artifacts. *Information and Software Technology* 163 (11 2023), 107299. doi:10.1016/j.infsof.2023.107299

[7] Virginia Braun and Victoria Clarke. 2006. Using Thematic Analysis in Psychology. *Qualitative Research in Psychology* 3, 2 (Jan. 2006), 77–101. doi:10.1191/1478088706qp063oa

[8] Victoria Clarke and Virginia Braun. 2017. Thematic analysis. *The Journal of Positive Psychology* 12 (5 2017), 297–298. Issue 3. doi:10.1080/17439760.2016.1262613

[9] Nicole Davila, Jorge Melegati, and Igor Wiese. 2024. Tales From the Trenches: Expectations and Challenges From Practice for Code Review in the Generative AI Era. *IEEE Software* 41 (11 2024), 38–45. Issue 6. doi:10.1109/MS.2024.3428439

[10] Michael Dorner, Maximilian Capraro, Oliver Treidler, Tom-Eric Kunz, Darja Šmite, Ehsan Zabardast, Daniel Mendez, and Krzysztof Wnuk. 2024. Taxing Collaborative Software Engineering. *IEEE Software* (2024), 1–8. doi:10.1109/MS.2023.3346646

[11] Michael Dorner, Daniel Mendez, Krzysztof Wnuk, Ehsan Zabardast, and Jacek Czerwonka. 2023. The Upper Bound of Information Diffusion in Code Review. *Empirical Software Engineering* (6 2023).

[12] M. E. Fagan. 1976. Design and code inspections to reduce errors in program development. *IBM Systems Journal* 15 (1976), 182–211. Issue 3. doi:10.1147/sj.153.0182

[13] Nargis Fatima, Sumaira Nazir, and Suriayati Chuprat. 2019. Knowledge Sharing, a Key Sustainable Practice Is on Risk: An Insight from Modern Code Review. In *2019 IEEE 6th International Conference on Engineering Technologies and Applied Sciences (ICETAS)*. IEEE, Kuala Lumpur, Malaysia, 1–6. doi:10.1109/ICETAS48360.2019.9117444

[14] Shane McIntosh, Yasutaka Kamei, Bram Adams, and Ahmed E. Hassan. 2016. An Empirical Study of the Impact of Modern Code Review Practices on Software Quality. *Empirical Software Engineering* 21, 5 (Oct. 2016), 2146–2189. doi:10.1007/s10664-015-9381-9

[15] Jaakko Sauvola, Sasu Tarkoma, Mika Klemettinen, Jukka Riekki, and David Doermann. 2024. Future of software development with generative AI. *Automated Software Engineering* 31, 1 (2024). doi:10.1007/s10515-024-00426-z

[16] Antero Taivalsaari, Tommi Mikkonen, and Cesare Pautasso. 2025. On the Future of Software Reuse in the Era of AI Native Software Engineering. *arXiv preprint arXiv:2508.19834* (2025).

[17] Laurie Williams. [n. d.]. Integrating pair programming into a software development process. In *Proceedings 14th Conference on Software Engineering Education and Training. 'In search of a software engineering profession' (Cat. No.PR01059)*. IEEE Comput. Soc, 27–36. doi:10.1109/CSEE.2001.913816

[18] Ehsan Zabardast, Javier Gonzalez-Huerta, and Binish Tanveer. 2022. Ownership vs Contribution: Investigating the Alignment Between Ownership and Contribution. In *2022 IEEE 19th International Conference on Software Architecture Companion (ICSA-C)*. IEEE, 30–34. doi:10.1109/ICSA-C54293.2022.00013